

CS 6476 Project 1

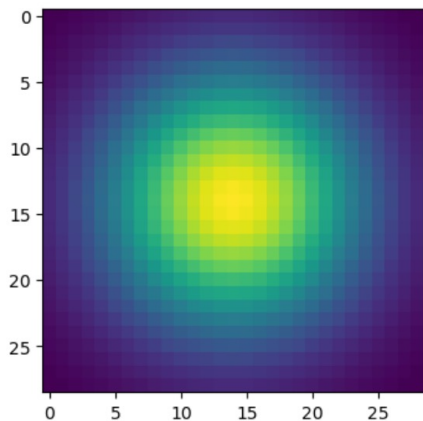
[Ruoran Jia]
[rjia41@gatech.edu]
[rjia41]
[903631225]

Part 1: Image filtering

[insert visualization of Gaussian kernel from project-1.ipynb here]



1D Gaussian Kernel



2D Gaussian Kernel

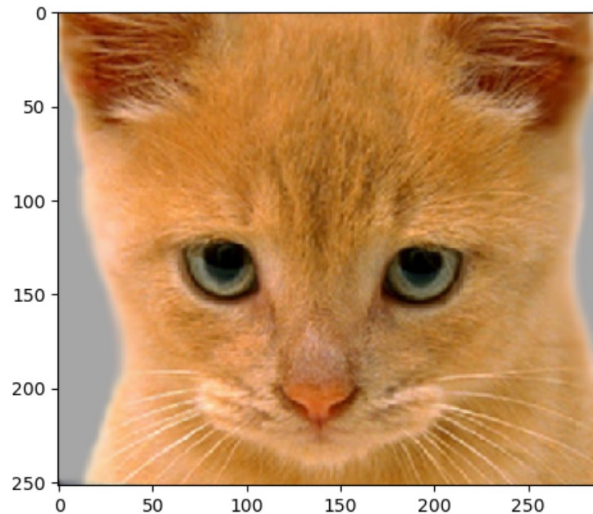
[Describe your implementation of `my_conv2d_numpy()` in words. Make sure to discuss padding, and the operations used between the filter and image.]

In `my_conv2d_numpy()` function, I first get the specific shape of image and filter, and use letters (`m,n,c` and `k,l`) to represent them. Then, I padded the image with 0.5 (round down) of the the height and width of the filter. After that, I create a loop that go through each row and column of the image to get the center pixel of each padding. I then use the filter by summing up the multiplication of kernel and filter, and finally replace the original number with the result I just compute.

Part 1: Image filtering

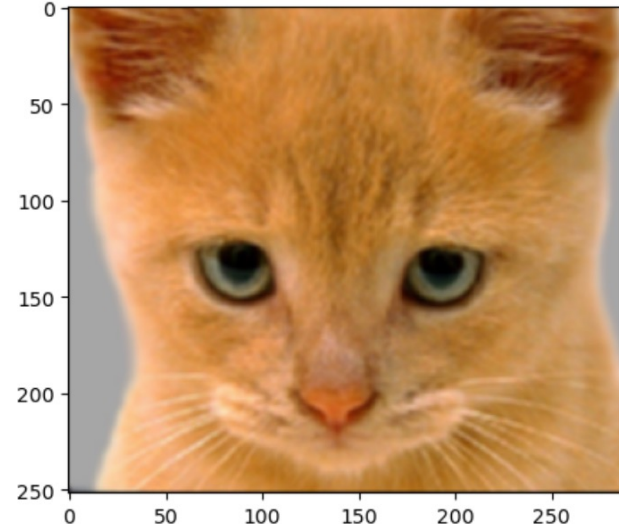
Identity filter

[insert the results from project-1.ipynb using
1b_cat.bmp with the identity filter here]



Small blur with a box filter

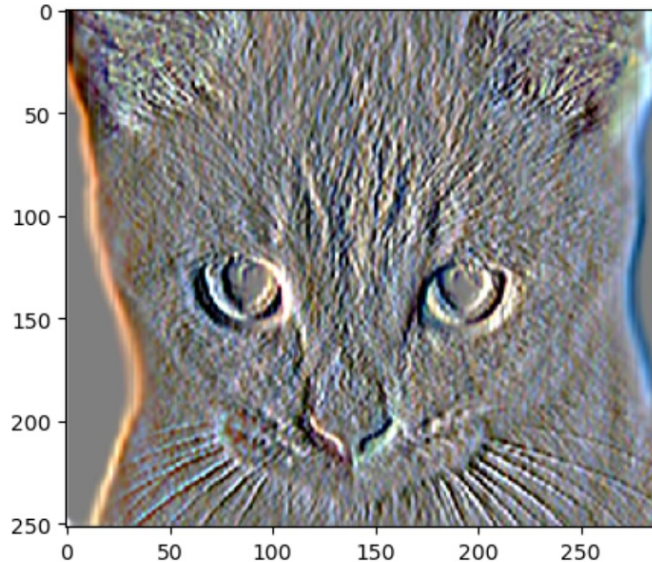
[insert the results from project-1.ipynb using
1b_cat.bmp with the box filter here]



Part 1: Image filtering

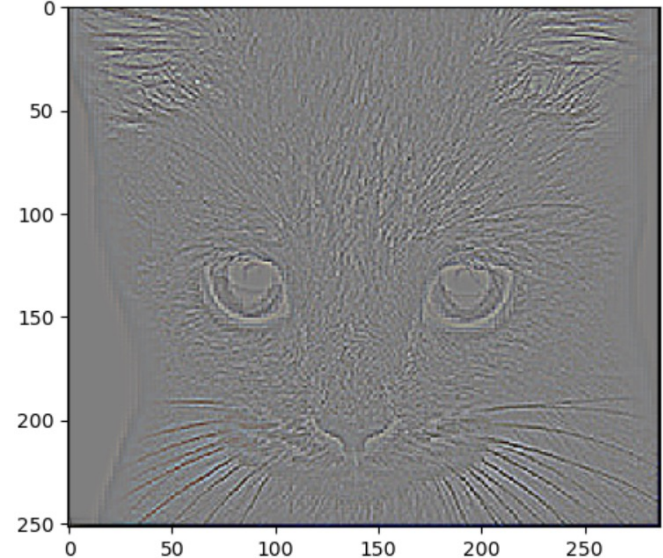
Sobel filter

[insert the results from project-1.ipynb using 1b_cat.bmp with the Sobel filter here]



Discrete Laplacian filter

[insert the results from project-1.ipynb using 1b_cat.bmp with the discrete Laplacian filter here]



Part 1: Hybrid images

[Describe the three main steps of `create_hybrid_image()` here. Explain how to ensure the output values are within the appropriate range for matplotlib visualizations.]

1. I first applied low frequency filter of image 1 by using `my_conv2d_numpy()` with the given filter.
2. I then subtract the image 2 by low frequency version of itself to get a high frequency version of image 2.
3. Finally, I add the two images I got from the above 2 steps to get a hybrid image, and use `np.clip()` to make sure that the result's pixel value is in the range (0,1)

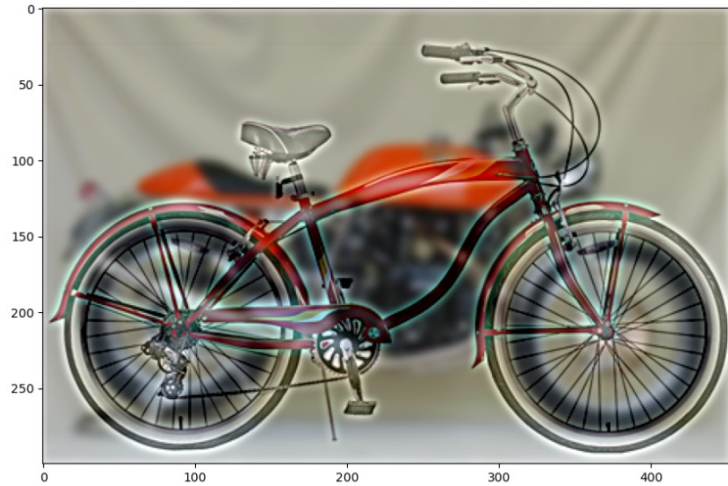
Cat + Dog



Cutoff frequency: 7

Part 1: Hybrid images

Motorcycle + Bicycle



Cutoff frequency: 3

Plane + Bird



Cutoff frequency: 2

Part 1: Hybrid images

Einstein + Marilyn



Cutoff frequency: 2

Submarine + Fish



Cutoff frequency: 3

Part 2: Hybrid images with PyTorch

Cat + Dog



Motorcycle + Bicycle



Part 2: Hybrid images with PyTorch

Plane + Bird



Einstein + Marilyn



Part 2: Hybrid images with PyTorch

Submarine + Fish



Part 1 vs. Part 2

[Compare the run-times of Parts 1 and 2 here, as calculated in project-1.ipynb. Which method is faster?]

Part 1: 5.015 seconds

Part 2: 0.930 seconds

Method 2 (using PyTorch) is faster.

Part 3: Understanding input/output shapes in PyTorch

[Consider a 1-channel 5x5 image and a 3x3 filter. What are the output dimensions of a convolution with the following parameters?

Stride = 1, padding = 0?

3x3

Stride = 2, padding = 0?

2x2

Stride = 1, padding = 1?

5x5

Stride = 2, padding = 1?]

3x3

[What are the input & output dimensions of the convolutions of the dog image and a 3x3 filter with the following parameters:

Dog image: (3, 361, 160)

Stride = 1, padding = 0

(3, 359, 408)

Stride = 2, padding = 0

(3, 180, 204)

Stride = 1, padding = 1

(3, 361, 410)

Stride = 2, padding = 1?]

(3, 181, 205)

Part 3: Understanding input/output shapes in PyTorch

[How many filters did we apply to the dog image?]

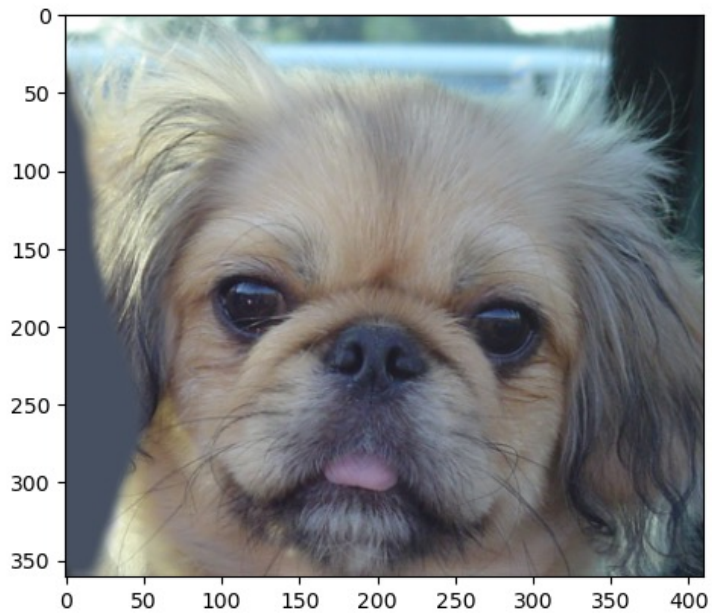
There are 12 filters in the filter bank.
There are 4 filters in one dimension, and there are 3 dimensions for the dog image in total.

[Section 3 of the handout gives equations to calculate output dimensions given filter size, stride, and padding. What is the intuition behind this equation?]

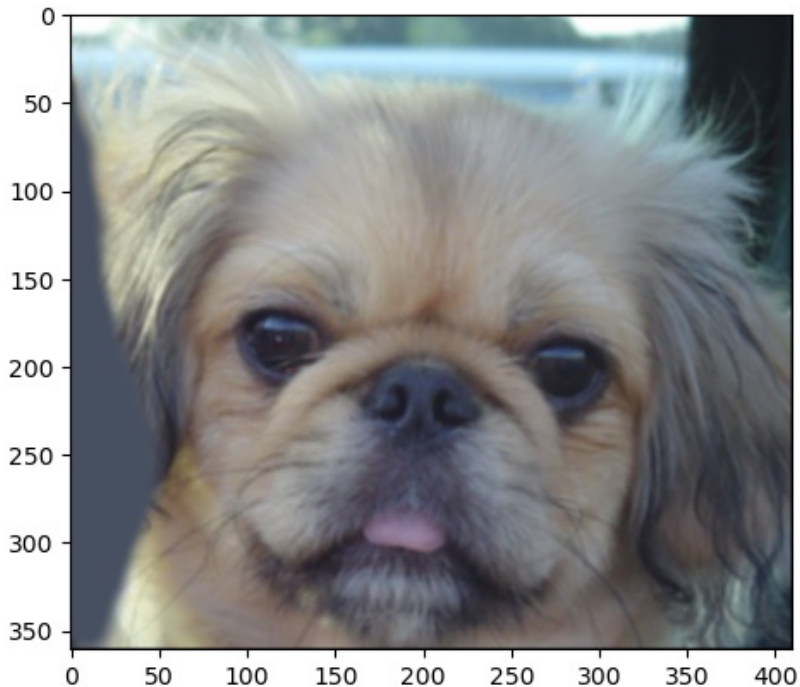
This equation is to help us calculate the height, weight (w_2 and h_2), and also dimension(d_2) of the output image, and we will first talk about how to calculate the w_2 and h_2 . We will use the padding and stride in the equation. For padding, as it will be padded on both side of the image, so we will add $2 * \text{padding}$ in the equation. For stride, if there is not enough pixel to let a pixel become center pixel, we use integer division of the stride then plus one. Besides, for the output dimension(d_2), it will be related to the kernel dimension: as group is used for generate different channel with different group of filters.

Part 3: Understanding input/output shapes in PyTorch

[insert visualization 0 here]

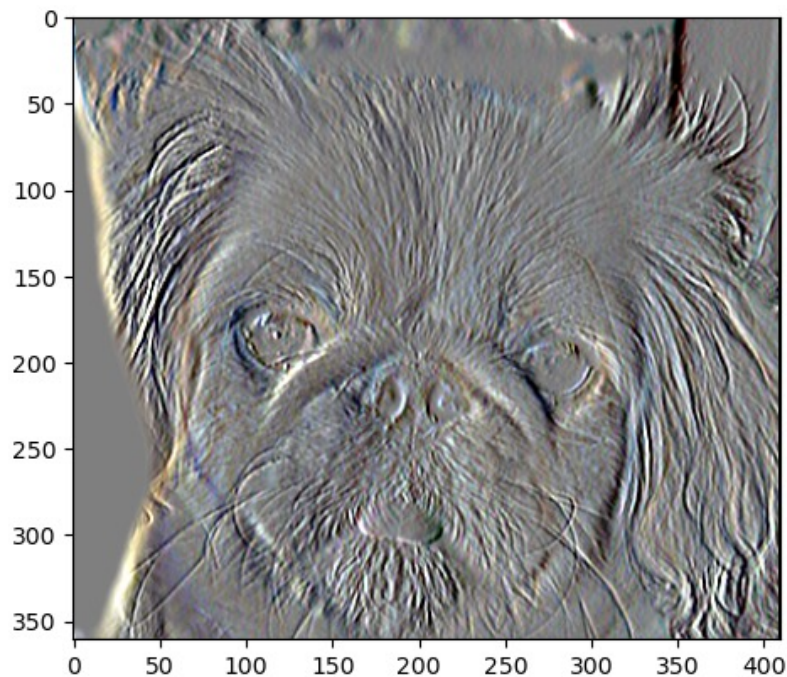


[insert visualization 1 here]

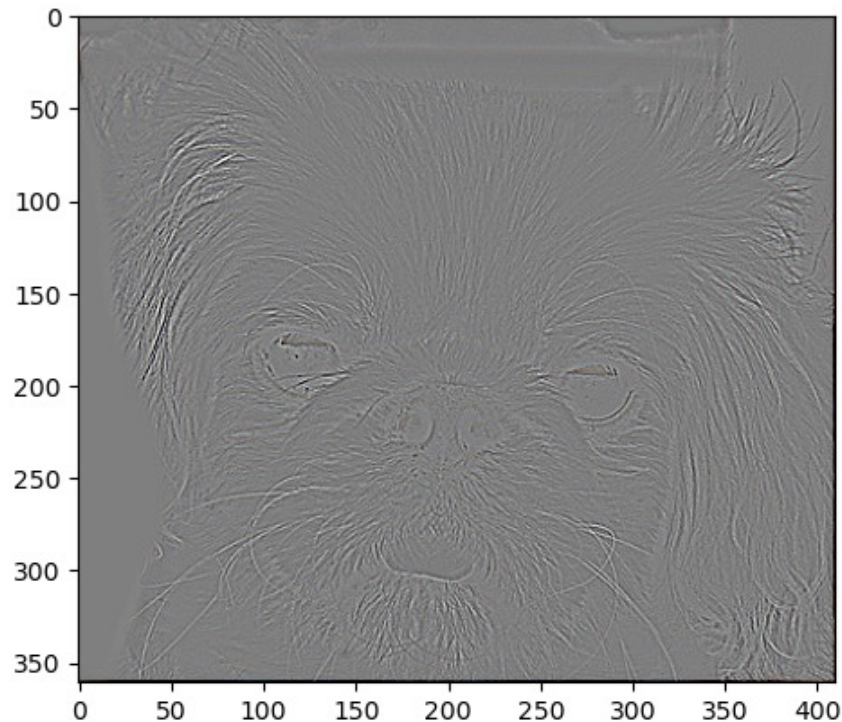


Part 3: Understanding input/output shapes in PyTorch

[insert visualization 2 here]



[insert visualization 3 here]



Part 4: Frequency Domain Convolutions

[Insert the visualizations of the dog image in the spatial and frequency domain]

[Insert the visualizations of the blurred dog image in the spatial and frequency domain]

Part 4: Frequency Domain Convolutions

[Insert the visualizations of the 2D Gaussian in the spatial and frequency domain]

[Why does our frequency domain representation of a Gaussian not look like a Gaussian itself?
How could we adjust the kernel to make these look more similar?]

Part 4: Frequency Domain Convolutions

[Briefly explain the Convolution Theorem and why this is related to deconvolution]

Part 4: Frequency Domain Convolutions

[Insert the visualizations of the mystery image in the spatial and frequency domain]

[Insert the visualizations of the mystery kernel in the spatial and frequency domain]

Part 4: Frequency Domain Convolutions

[Insert the de-blurred mystery image and its visualizations in the spatial and frequency domain]

[Insert the de-blurred mystery image and its visualizations in the spatial and frequency domain after adding salt and pepper noise]

Part 4: Frequency Domain Convolutions

[What factors limit the potential uses of deconvolution in the real world? Give two possible factors]

[We performed two convolutions of the dog image with the same Gaussian (one in the spatial domain, one in the frequency domain). How do the two compare, and why might they be different?]

Conclusion

[How does varying the cutoff frequency value or swapping images within a pair influences the resulting hybrid image?]

With higher cutoff frequency, images with higher frequencies will have clearer edges. With lower cutoff frequency, images with higher frequencies will have blur edges. By swapping images within a pair, image with higher frequency is sharpened even more and images with lower image with lower frequency is blurred more, thus will not have the same effect after hybrid (example with cat + dog hybrid image is showed in the next page). In addition, I found the best cutoff frequency for the cat + dog hybrid image is 5.